

What is Android?

Android is an **open-source**, Linux-based mobile operating system developed by **Google** for smartphones, tablets, smart TVs, smartwatches, automotive systems, and other smart devices. It provides a complete software stack that includes an operating system, middleware, application framework, and built-in applications.

Android allows developers to create mobile applications using programming languages such as **Java**, **Kotlin**, and **C++ (through the Android NDK)**. In this course, we will primarily use **Java** for Android application development.

The first commercial Android device, the **HTC Dream (T-Mobile G1)**, was released in September 2008. Since then, Android has become the world's most widely used mobile operating system because of its flexibility, open-source nature, and extensive application ecosystem.

Android applications are primarily distributed through the **Google Play Store**, although users can also install applications from other trusted sources.

Features of Android

Android offers a wide range of features that make it one of the most powerful mobile operating systems available today.

1. Open Source
2. Linux-based Operating System
3. Multitasking
4. Touch Based User Interface
5. Rich Multimedia Support
6. Connectivity Options
7. Database Support
8. Security
9. Notification System
10. Sensor Support
11. Google Services Integration

History of Android

The history of Android is a remarkable journey from a **niche digital camera software to the world's most dominant operating system**, powering over 3.9 billion active devices.

1. The Foundation (2003–2005)

Android Inc. was founded in October 2003 in Palo Alto, California, by **Andy Rubin, Rich Miner, Nick Sears, and Chris White**.

- **The Camera Plan:** Initially, the team intended to build an advanced operating system for digital cameras.
- **The Pivot:** Realising the digital camera market was too small, they pivoted to target mobile phones.
- **The Google Acquisition:** Facing financial strain, the startup was **acquired by Google in August 2005 for \$50 million**. The original founders moved to Google to continue building the OS on a Linux foundation.

2. The Launch & Early Evolution (2007–2010)

When Apple unveiled the touch-centric iPhone in 2007, the Android team had to completely rework their prototype, which originally resembled a BlackBerry with a physical keyboard.

- **Open Handset Alliance (OHA):** In November 2007, Google announced a consortium of tech giants—including Samsung, HTC, and Intel—to develop a free, open-source mobile platform.
- **The First Phone:** On **September 23, 2008**, the first commercial Android smartphone, the **T-Mobile G1 (also known as the HTC Dream)**, made its debut. It featured a physical slide-out keyboard, a trackball, and the Android 1.0 software.

3. The "Sweet Treat" Eras

Starting with Android 1.5, Google began naming its major releases after popular desserts in alphabetical order. This quirky naming convention became a hallmark of the tech industry.

Android Version	Major Features Introduced	Android Version	Major Features Introduced
Cupcake	Virtual Keyboard, Widgets	Android 11	One-Time Permissions
Donut	Universal Search	Android 12	Material You
Eclair	Google Maps Navigation	Android 13	Notification Permission
Froyo	Wi-Fi Hotspot	Android 14	Predictive Back Gesture
Gingerbread	NFC Support	Android 15	Large-Screen Enhancements
Honeycomb	Tablet Interface		
Ice Cream Sandwich	Face Unlock		
Jelly Bean	Google Now, Project Butter		
KitKat	Optimized for Low-End Devices		
Lollipop	Material Design, ART		
Marshmallow	Runtime Permissions		
Nougat	Split Screen		
Oreo	Notification Channels, Picture-in-Picture		
Pie	Gesture Navigation		
Android 10	Dark Theme		

The Necessary Tools

Android Studio

Android Studio is an integrated development environment used for Android development that is based on the popular IntelliJ IDEA.

- An install of Android Studio includes:
- Android SDK: the latest version of the Android SDK
- Android SDK tools and platform tools: tools for debugging and testing your apps
- A system image for the Android emulator: lets you create and test your apps on different virtual devices

Creating an Android Project



Creating a new application

Create New Project

 **New Project**
Android Studio

Configure your new project

Application name:

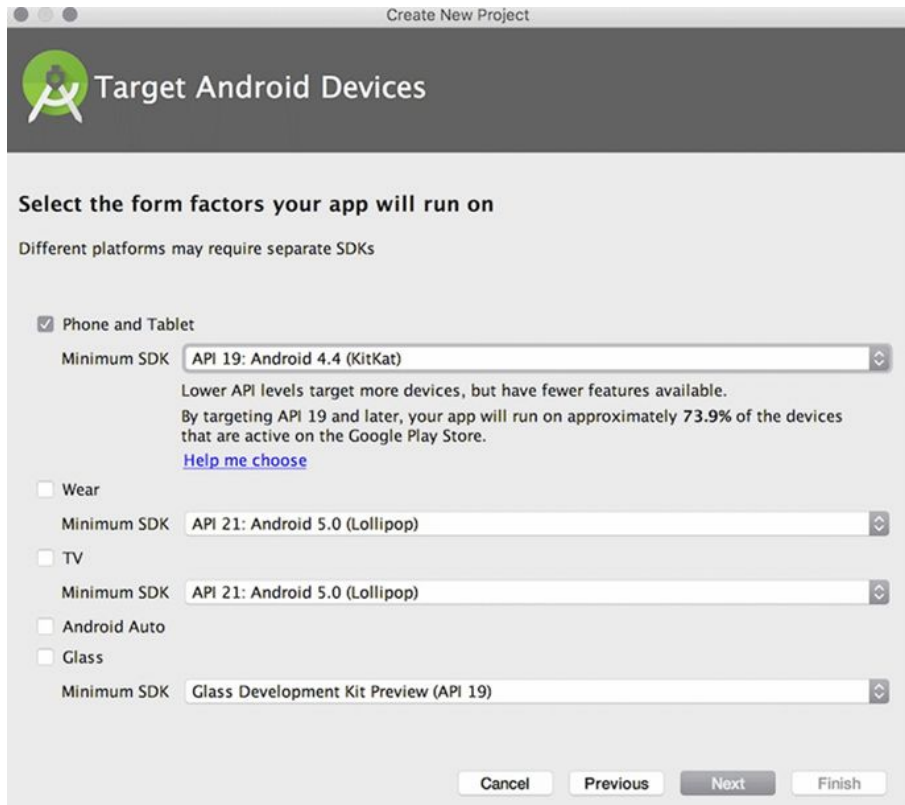
Company Domain:

Package name: [Edit](#)

☐ Include C++ Support

Project location:

Specifying device support



Create New Project

 Target Android Devices

Select the form factors your app will run on

Different platforms may require separate SDKs

☒ Phone and Tablet

Minimum SDK

Lower API levels target more devices, but have fewer features available.
By targeting API 19 and later, your app will run on approximately 73.9% of the devices that are active on the Google Play Store.
[Help me choose](#)

☐ Wear

Minimum SDK

☐ TV

Minimum SDK

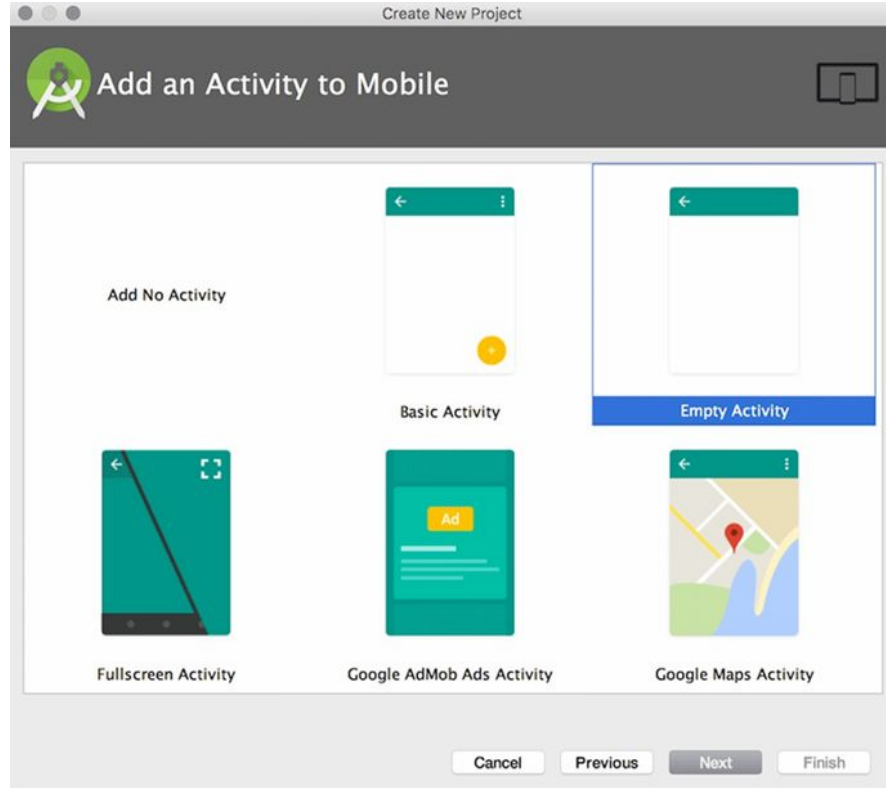
☐ Android Auto

☐ Glass

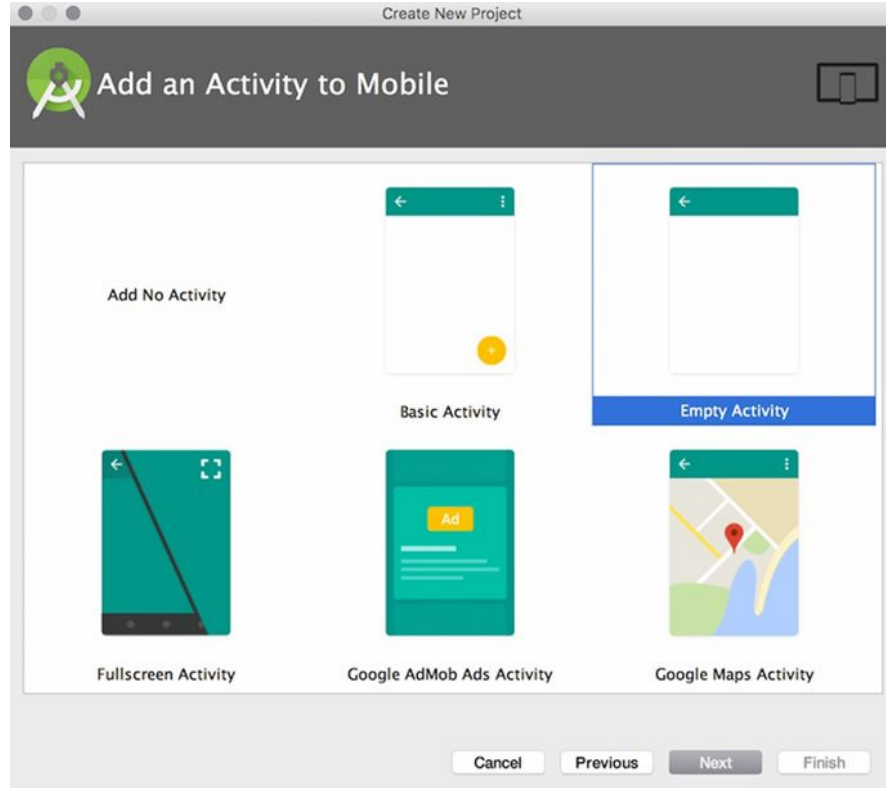
Minimum SDK

Cancel Previous Next Finish

Choosing a type of activity



Choosing a type of activity



Configuring the new activity

Create New Project

 Customize the Activity 

Creates a new empty activity



Empty Activity

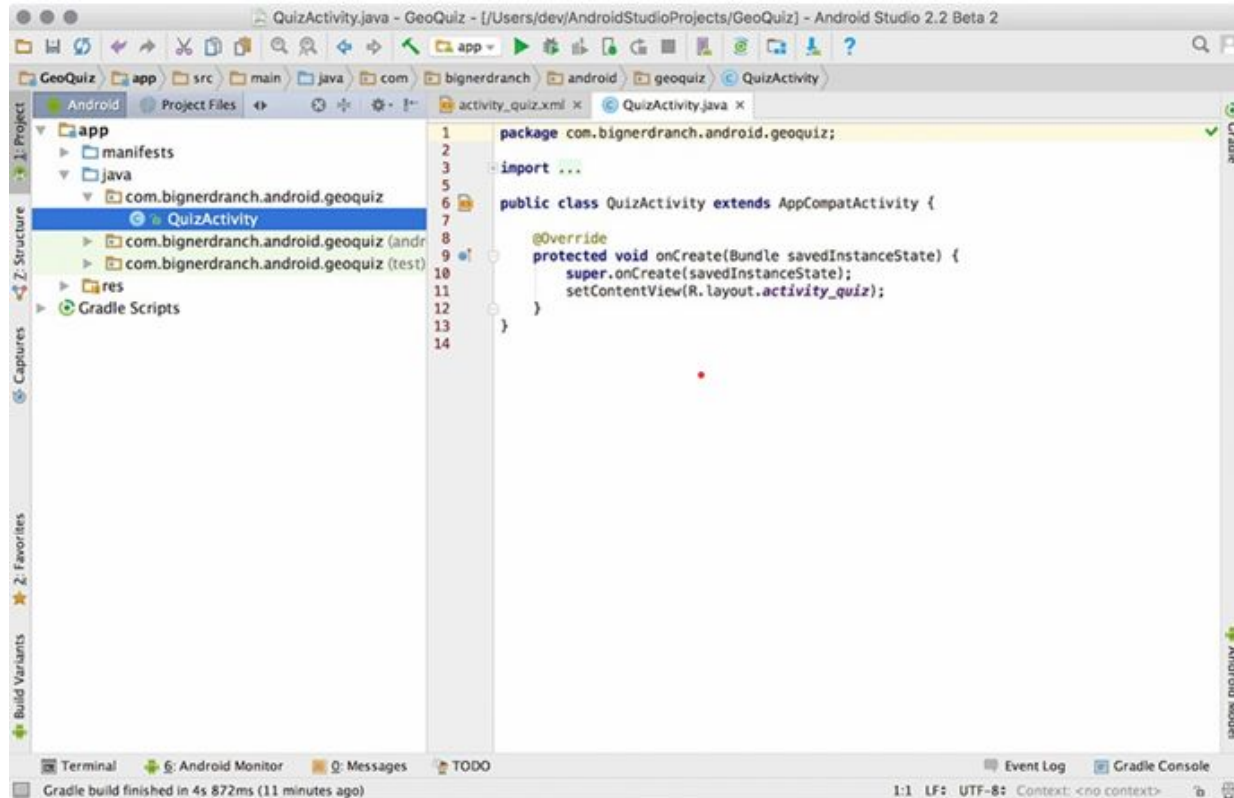
Activity Name:

☒ Generate Layout File

Layout Name:

☒ Backwards Compatibility (AppCompat)

A Fresh project window



Four main pillars of Android

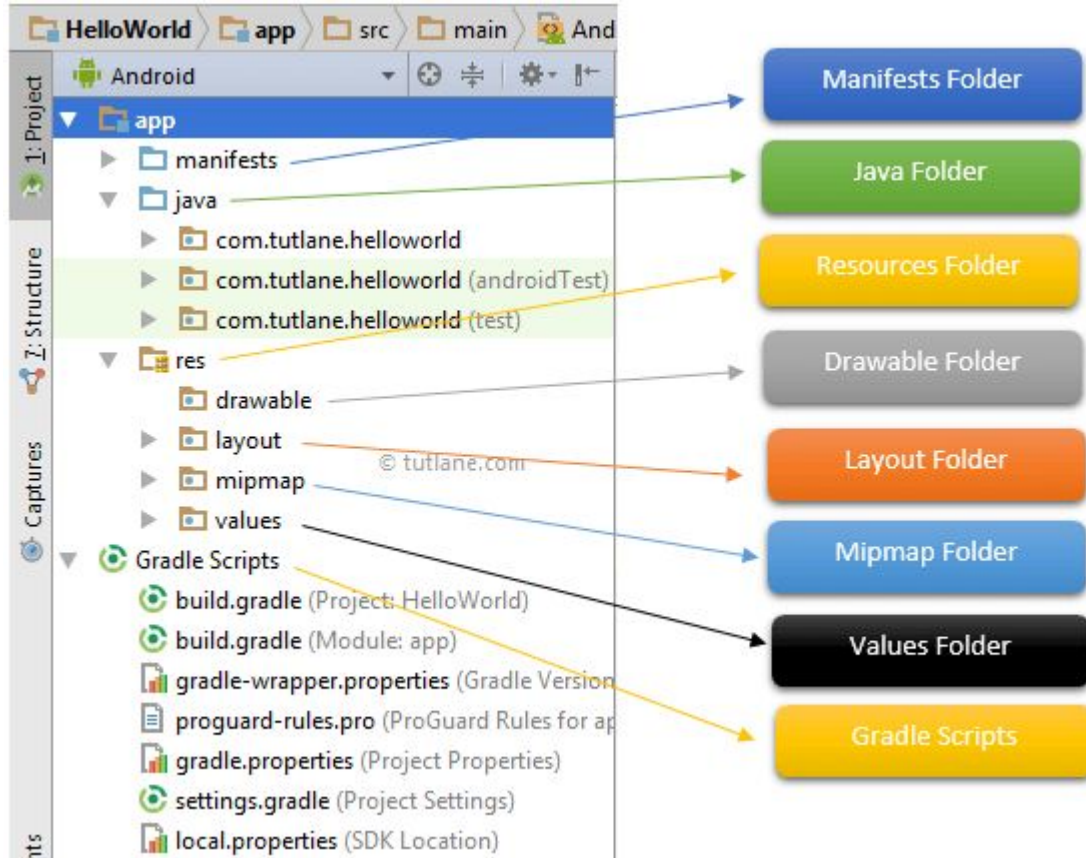
Activities: You can understand a little bit by seeing its name. The activity represents your app. The activity makes your app communicate with the user. Activity let the user use your app, which means all the screen, buttons, switch, and views are the part of the activity that helps to share your idea with the user. Users will see the app with the help of an activity. Shortly you can say it as “user interface”.

Services: Services are this part which users can't see directly but It's very important and one of the main parts of your app. Service runs in the background to perform long-running operations even if the users are not active and the application is destroyed. A service interacts with a content provider, performs multiplayer in the background threads.

Broadcast Receivers: Broadcast receivers help your app to communicate with android systems. With the help of a broadcast receiver, your app can communicate with other app events and system events. One has to be very aware of broadcast receivers. It can help to build a fast and user-friendly app by communicating with the system.

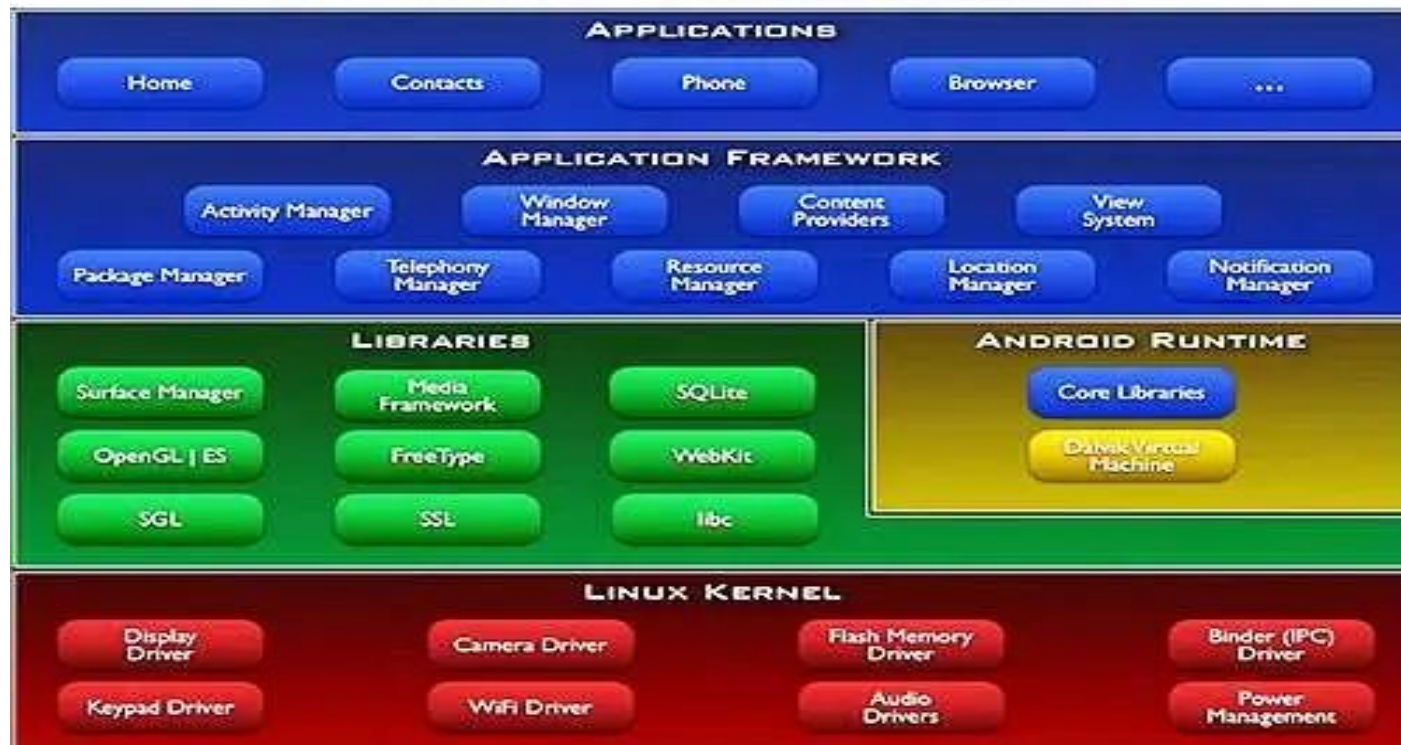
Content Providers: Content Provider is a very important part of android. It handles the data part of your app. It helps to provide the main content of your app with the main data that you want to show in your app. Not only you can access your app data you can also access the data of other apps as per your need and app permissions.

Basic structural layout of Android Studio



Android Platform Architecture

Android™ Architecture



The main components of android architecture are following:-

- Applications
- Application Framework
- Android Runtime
- Platform Libraries
- Linux Kernel

Applications –

Applications is the top layer of android architecture. The pre-installed applications like home, contacts, camera, gallery etc and third party applications downloaded from the play store like chat applications, games etc. will be installed on this layer only.

It runs within the Android run time with the help of the classes and services provided by the application framework.

Application framework –

Application Framework provides several important classes which are used to create an Android application. It provides a generic abstraction for hardware access and also helps in managing the user interface with application resources. Generally, it provides the services with the help of which we can create a particular class and make that class helpful for the Applications creation.

It includes different types of services activity manager, notification manager, view system, package manager etc. which are helpful for the development of our application according to the prerequisite.

Application runtime –

Android Runtime environment is one of the most important part of Android. It contains components like core libraries and the Dalvik virtual machine(DVM). Mainly, it provides the base for the application framework and powers our application with the help of the core libraries.

Like Java Virtual Machine (JVM), **Dalvik Virtual Machine (DVM)** is a register-based virtual machine and specially designed and optimized for android to ensure that a device can run multiple instances efficiently. It depends on the layer Linux kernel for threading and low-level memory management. The core libraries enable us to implement android applications using the standard JAVA or Kotlin programming languages.

Native C/C++ libraries

Many core Android system components and services, such as ART and HAL, are built from native code that requires native libraries written in C and C++. The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to apps. For example, you can access OpenGL ES through the Android framework's Java OpenGL API to add support for drawing and manipulating 2D and 3D graphics in your app.

If you are developing an app that requires C or C++ code, you can use the Android NDK to access some of these native platform libraries directly from your native code.

Java API framework

The entire feature-set of the Android OS is available to you through APIs written in the Java language. These APIs form the building blocks you need to create Android apps by simplifying the reuse of core, modular system components and services, which include the following:

- A rich and extensible view system you can use to build an app's UI, including lists, grids, text boxes, buttons, and even an embeddable web browser
- A resource manager, providing access to non-code resources such as localized strings, graphics, and layout files
- A notification manager that enables all apps to display custom alerts in the status bar
- An activity manager that manages the lifecycle of apps and provides a common navigation back stack
- Content providers that enable apps to access data from other apps, such as the Contacts app, or to share their own data. Developers have full access to the same framework APIs that Android system apps use.

System apps

Android comes with a set of core apps for email, SMS messaging, calendars, internet browsing, contacts, and more. Apps included with the platform have no special status among the apps the user chooses to install. So, a third-party app can become the user's default web browser, SMS messenger, or even the default keyboard. Some exceptions apply, such as the system's Settings app.

The system apps function both as apps for users and to provide key capabilities that developers can access from their own app. For example, if you want your app to deliver SMS messages, you don't need to build that functionality yourself. You can instead invoke whichever SMS app is already installed to deliver a message to the recipient you specify.

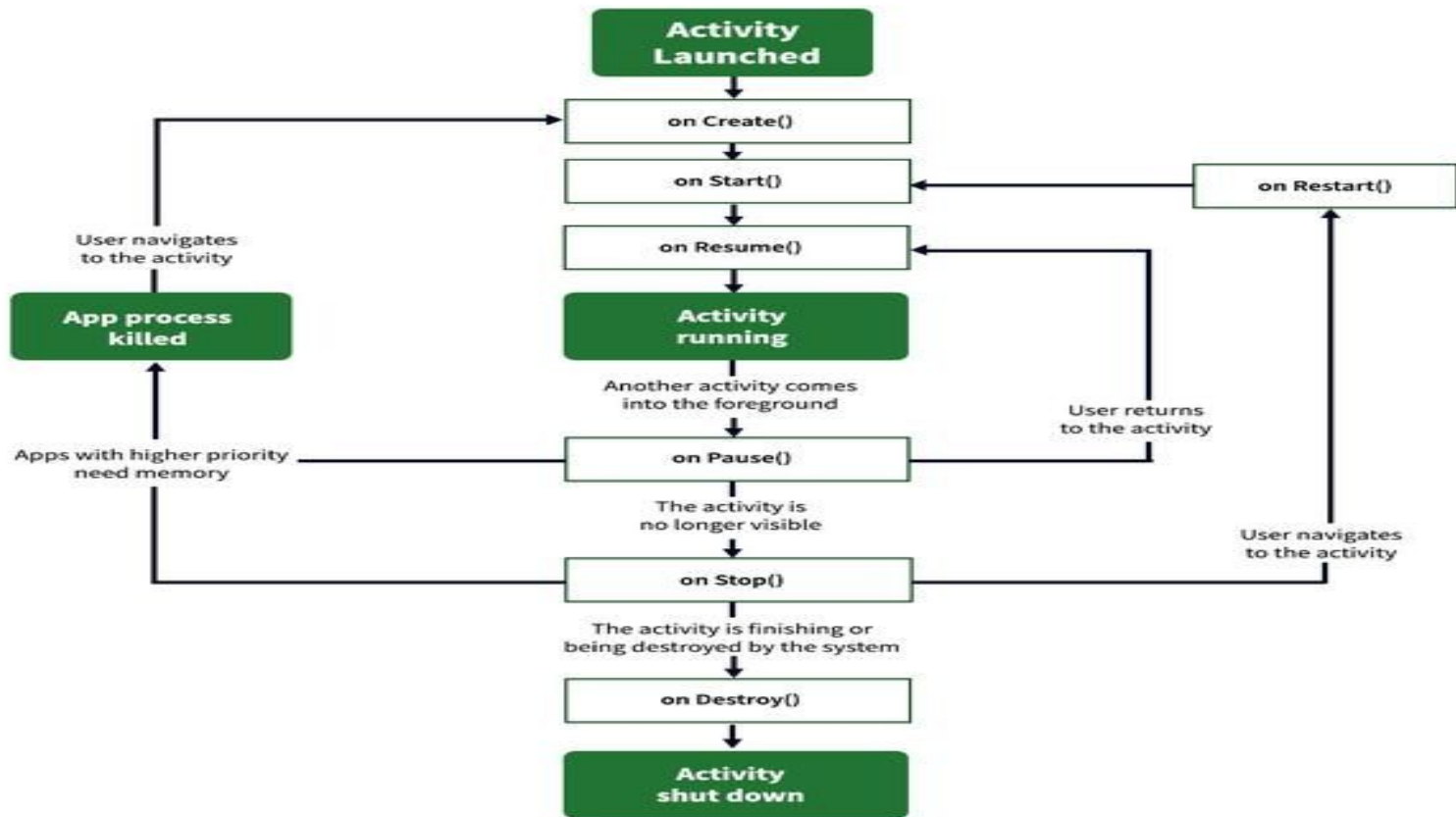
Important Android Terminology

Context : The context is the central command center for an Android application. All application-specific functionality can be accessed through the context.

Activity : An Android application is a collection of tasks, each of which is called an Activity. Each Activity within an application has a unique task or purpose.

Intent : The Android operating system uses an asynchronous messaging mechanism to match task requests with the appropriate Activity. Each request is packaged as an Intent. You can think of each such request as a message stating *an intent to do something*.

Service: Tasks that do not require user interaction can be encapsulated in a service. A service is most useful when the operation is lengthy or needs to be done regularly (such as checking server for new email).



Activity Lifecycle in Android

1. onCreate()

It is called when the activity is first created. This is where all the static work is done like creating views, binding data to lists, etc. This method also provides a Bundle containing its previous frozen state, if there was one.

2. onStart()

It is invoked when the activity is visible to the user. It is followed by onResume() if the activity is invoked from the background. It is also invoked after onCreate() when the activity is first started.

3. onRestart()

It is invoked after the activity has been stopped and prior to its starting stage and thus is always followed by onStart() when any activity is revived from background to on-screen.

4. onResume()

It is invoked when the activity starts interacting with the user. At this point, the activity is at the top of the activity stack, with a user interacting with it. Always followed by onPause() when the activity goes into the background or is closed by the user.

5. onPause()

It is invoked when an activity is going into the background but has not yet been killed. It is a counterpart to onResume(). When an activity is launched in front of another activity, this callback will be invoked on the top activity (currently on screen). The activity, under the active activity, will not be created until the active activity's onPause() returns, so it is recommended that heavy processing should not be done in this part.

6. onStop()

It is invoked when the activity is not visible to the user. It is followed by **onRestart()** when the activity is revoked from the background, followed by onDestroy() when the activity is closed or finished, and nothing when the activity remains on the background only. Note that this method may never be called, in low memory situations where the system does not have enough memory to keep the activity's process running after its onPause() method is called.

7. onDestroy()

The final call received before the activity is destroyed. This can happen either because the activity is finishing (when finish() is invoked) or because the system is temporarily destroying this instance of the activity to save space. To distinguish between these scenarios, check it with **isFinishing()** method.

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Toast;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toast toast = Toast.makeText(getApplicationContext(), "onCreate Called",
Toast.LENGTH_LONG).show();
    }
    protected void onStart() {
        super.onStart();
        Toast toast = Toast.makeText(getApplicationContext(), "onStart Called",
Toast.LENGTH_LONG).show();
    }
    @Override
    protected void onRestart() {
        super.onRestart();
        Toast toast = Toast.makeText(getApplicationContext(), "onRestart Called",
Toast.LENGTH_LONG).show();
    }
}
```

```
protected void onPause() {  
    super.onPause();  
    Toast toast = Toast.makeText(getApplicationContext(), "onPause Called",  
    Toast.LENGTH_LONG).show();  
}  
  
    protected void onResume() {  
        super.onResume();  
        Toast toast = Toast.makeText(getApplicationContext(), "onResume Called",  
    Toast.LENGTH_LONG).show();  
    }  
  
    protected void onStop() {  
        super.onStop();  
        Toast toast = Toast.makeText(getApplicationContext(), "onStop Called",  
    Toast.LENGTH_LONG).show();  
    }  
  
    protected void onDestroy() {  
        super.onDestroy();  
        Toast toast = Toast.makeText(getApplicationContext(), "onDestroy Called",  
    Toast.LENGTH_LONG).show();  
    }  
}
```

Intents and Intent filters

An intent is to perform an action on the screen.

It is mostly used to start activity, send broadcast receiver, start services and send message between two activities.

It is a messaging object which tells what kind of action to be performed.

The intent's most significant use is the launching of the activity.

Body of Intent:

action: The general action to be performed, such as ACTION_VIEW, ACTION_EDIT, ACTION_MAIN, etc.

data: The data to operate on, such as a person record in the contacts database, expressed as a Uri

Basically two intents are there in android.

- Implicit Intents

- Explicit Intents

Types of Intents

- **Implicit intents** do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it. For example, if you want to show the user a location on a map, you can use an implicit intent to request that another capable app show a specified location on a map.
- **Explicit intents** specify which application will satisfy the intent, by supplying either the target app's package name or a fully-qualified component class name. You'll typically use an explicit intent to start a component in your own app, because you know the class name of the activity or service you want to start. For example, you might start a new activity within your app in response to a user action, or start a service to download a file in the background.

Implicit Intent

syntax:

```
Intent intent=new Intent(Intent.ACTION_VIEW);  
intent.setData(Uri.parse("https://www.google.com/"));  
startActivity(intent);
```

Explicit Intent

Syntax:

```
Intent i = new Intent(getApplicationContext(), ActivityTwo.class);  
startActivity(i);
```

Intent filter

An app component should declare separate filters for each unique job it can do. For example, one activity in an image gallery app may have two filters: one filter to view an image, and another filter to edit an image.

When the activity starts, it inspects the Intent and decides how to behave based on the information in the Intent (such as to show the editor controls or not).

Each intent filter is defined by an `<intent-filter>` element in the app's manifest file, nested in the corresponding app component (such as an `<activity>` element).

```
<activity android:name="MainActivity" android:exported="true">
  <!-- This activity is the main entry, should appear in app launcher -->
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```

```
<activity android:name="ShareActivity" android:exported="false">
  <!-- This activity handles "SEND" actions with text data -->
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="text/plain"/>
  </intent-filter>
  <!-- This activity also handles "SEND" and "SEND_MULTIPLE" with media data -->
  <intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <action android:name="android.intent.action.SEND_MULTIPLE"/>
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:mimeType="application/vnd.google.panorama360+jpg"/>
    <data android:mimeType="image/*"/>
    <data android:mimeType="video/*"/>
  </intent-filter>
</activity>
```

Elements of Intent filter

Inside the <intent-filter>, you can specify the type of intents to accept using one or more of these three elements:

<action>

Declares the intent action accepted, in the **name** attribute. The value must be the literal string value of an action, not the class constant.

<data>

Declares the type of data accepted, using one or more attributes that specify various aspects of the data URI (**scheme**, **host**, **port**, **path**) and MIME type.

<category>

Declares the intent category accepted, in the **name** attribute. The value must be the literal string value of an action, not the class constant.

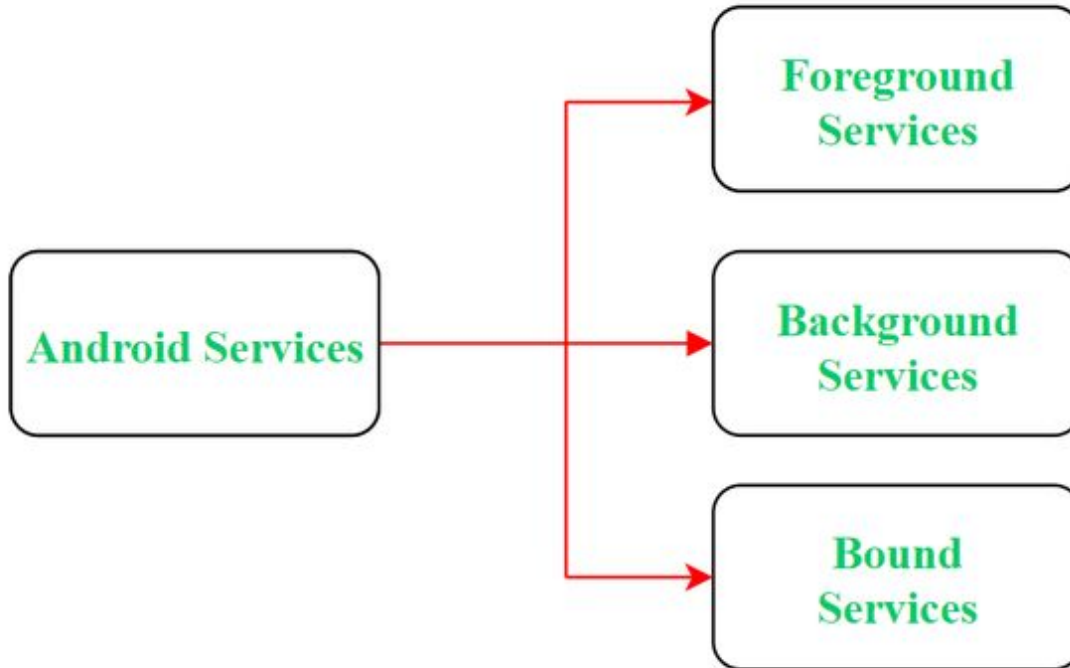
Services

Services in Android are a special component that facilitates an application to run in the background in order to perform long-running operation tasks.

The prime aim of a service is to ensure that the application remains active in the background so that the user can operate multiple applications at the same time.

application components can bind itself to service to carry out **inter-process communication(IPC)**.

Types of Services



1. Foreground Services:

Services that notify the user about its ongoing operations are termed as Foreground Services. Users can interact with the service by the notifications provided about the ongoing task. Such as in downloading a file, the user can keep track of the progress in downloading and can also pause and resume the process.

2. Background Services:

Background services do not require any user intervention. These services do not notify the user about ongoing background tasks and users also cannot access them. The process like schedule syncing of data or storing of data fall under this service.

Types of services

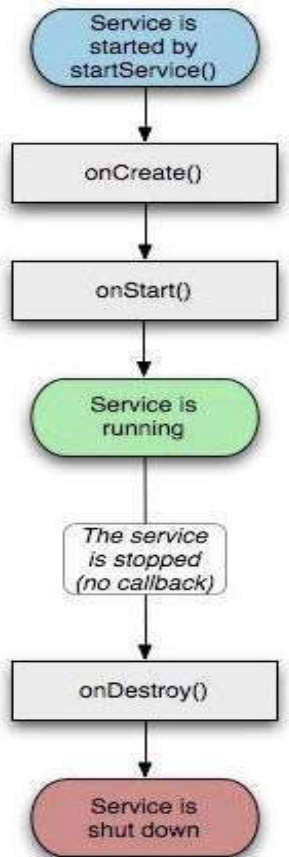
1. Started Service (Unbounded Service):

By following this path, a service will initiate when an application component calls the **startService()** method. Once initiated, the service can run continuously in the background even if the component is destroyed which was responsible for the start of the service. Two options are available to stop the execution of service:

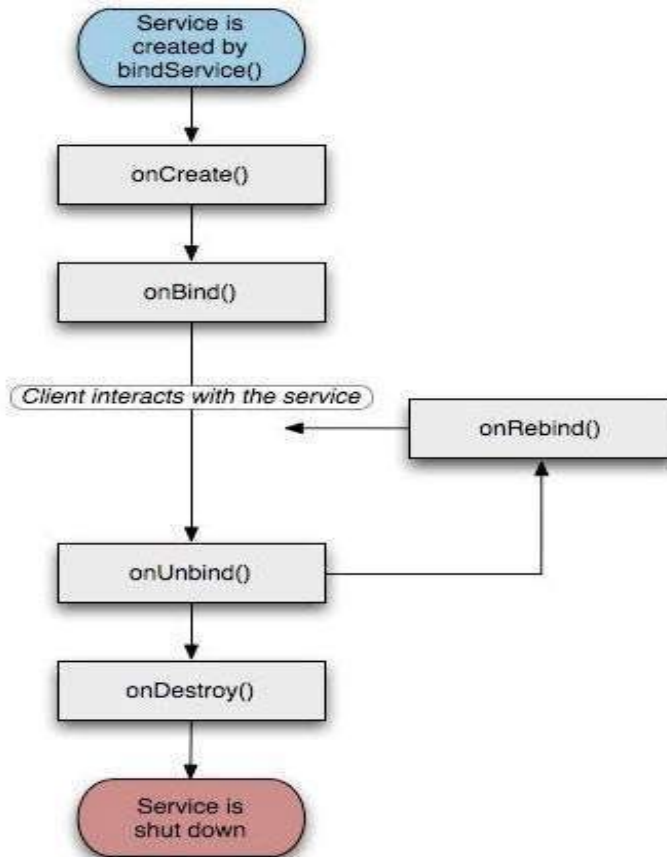
- By calling **stopService()** method,
- The service can stop itself by using **stopSelf()** method.

2. Bounded Service:

It can be treated as a server in a client-server interface. By following this path, android application components can send requests to the service and can fetch results. A service is termed as bounded when an application component binds itself with a service by calling **bindService()** method. To stop the execution of this service, all the components must unbind themselves from the service by using **unbindService()** method.



UN Bounded Service



Bounded services

Fundamentals of android services

`onStartCommand()`:

The Android service calls this method when a component(eg: activity) requests to start a service using `startService()`. Once the service is started, it can be stopped explicitly using `stopService()` or `stopSelf()` methods.

`onBind()`:

This method is mandatory to implement in android service and is invoked whenever an application component calls the `bindService()` method in order to bind itself with a service. User-interface is also provided to communicate with the service effectively by returning an `IBinder` object. If the binding of service is not required then the method must return null.

onUnbind():

The Android system invokes this method when all the clients get disconnected from a particular service interface.

onRebind():

Once all clients are disconnected from the particular interface of service and there is a need to connect the service with new clients, the system calls this method.

onCreate()

Whenever a service is created either using onStartCommand() or onBind(), the android system calls this method. This method is necessary to perform a one-time-setup.

onDestroy()

When a service is no longer in use, the system invokes this method just before the service destroys as a final clean up call. Services must implement this method in order to clean up resources like registered listeners, threads, receivers, etc.

Broadcast Receivers

Broadcast Receivers simply respond to broadcast messages from other applications or from the system itself. These messages are sometime called events or intents. For example, applications can also initiate broadcasts to let other applications know that some data has been downloaded to the device and is available for them to use, so this is broadcast receiver who will intercept this communication and will initiate appropriate action.

There are following two important steps to make BroadcastReceiver works for the system broadcast intents –

1. Creating the Broadcast Receiver.
2. Registering Broadcast Receiver

Creating the Broadcast Receiver

```
public class MyReceiver extends BroadcastReceiver {
```

```
    @Override
```

```
    public void onReceive(Context context, Intent intent) {
```

```
        Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();
```

```
    }
```

```
}
```

Registering Broadcast Receiver

```
<application
```

```
    android:icon="@drawable/ic_launcher"
```

```
    android:label="@string/app_name"
```

```
    android:theme="@style/AppTheme" >
```

```
    <receiver android:name="MyReceiver">
```

```
        <intent-filter>
```

```
            <action android:name="android.intent.action.BOOT_COMPLETED"/>
```

```
        </intent-filter>
```

```
    </receiver>
```

```
</application>
```

System Event Constants

android.intent.action.BATTERY_CHANGED:- Sticky broadcast containing the charging state, level, and other information about the battery.

android.intent.action.BATTERY_LOW:- Indicates low battery condition on the device.

android.intent.action.BATTERY_OKAY:- Indicates the battery is now okay after being low.

android.intent.action.BOOT_COMPLETED:- This is broadcast once, after the system has finished booting.

android.intent.action.BUG_REPORT:- Show activity for reporting a bug.

android.intent.action.CALL:- Perform a call to someone specified by the data.

android.intent.action.CALL_BUTTON:-The user pressed the "call" button to go to the dialer or other appropriate UI for placing a call.

android.intent.action.DATE_CHANGED:-The date has changed.

android.intent.action.REBOOT:- Have the device reboot.

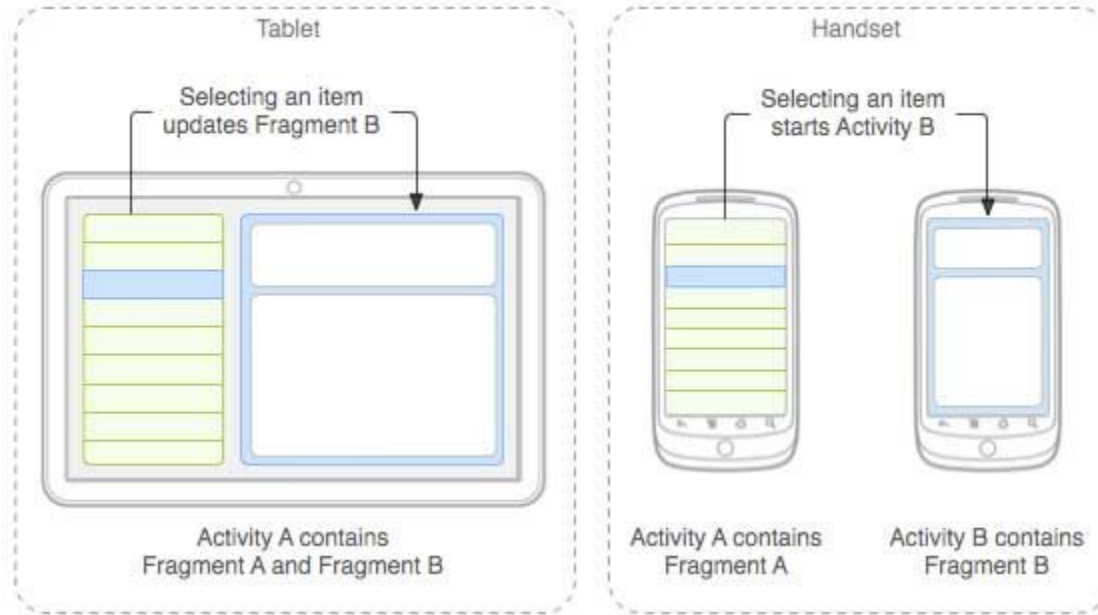
Fragments

A Fragment is a piece of an activity which enable more modular activity design. It will not be wrong if we say, a fragment is a kind of sub-activity.

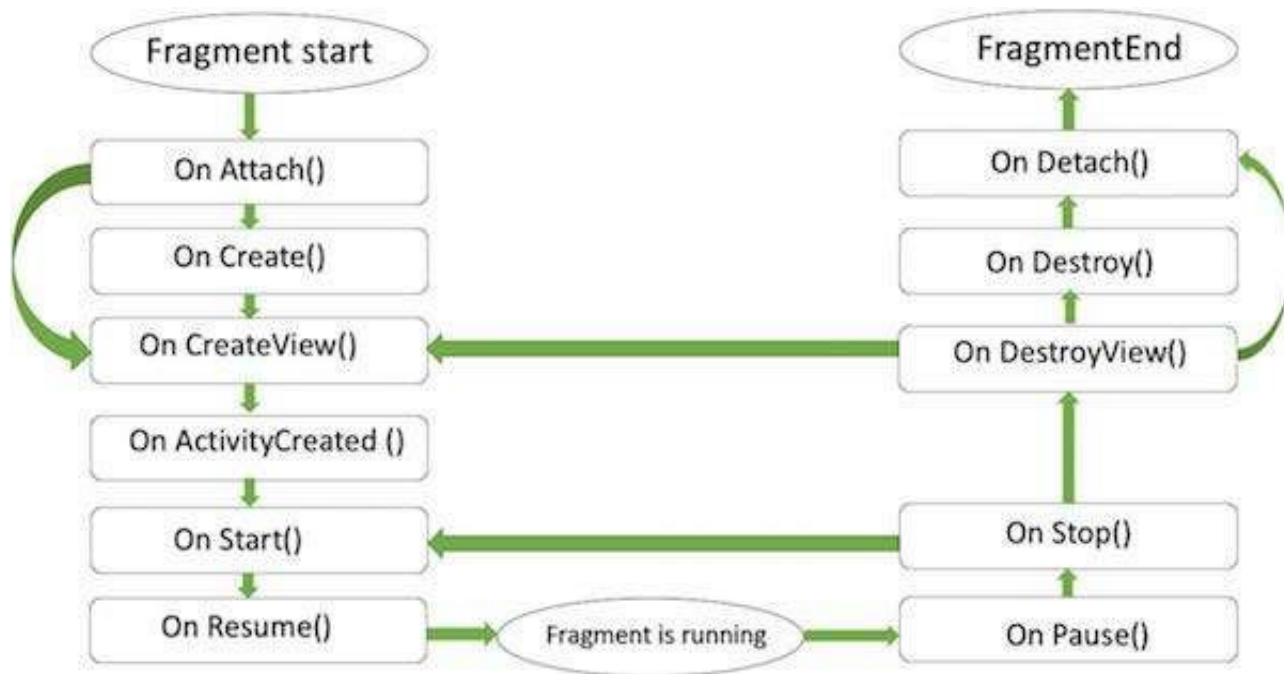
Following are important points about fragment –

- A fragment has its own layout and its own behaviour with its own life cycle callbacks.
- You can add or remove fragments in an activity while the activity is running.
- You can combine multiple fragments in a single activity to build a multi-pane UI.
- A fragment can be used in multiple activities.
- Fragment life cycle is closely related to the life cycle of its host activity which means when the activity is paused, all the fragments available in the activity will also be stopped.
- A fragment can implement a behaviour that has no user interface component.
- Fragments were added to the Android API in Honeycomb version of Android which API version 11.

Following is a typical example of how two UI modules defined by fragments can be combined into one activity for a tablet design, but separated for a handset design.



Fragment Lifecycle



Here is the list of methods which you can to override in your fragment class –

onAttach() The fragment instance is associated with an activity instance. The fragment and the activity is not fully initialized. Typically you get in this method a reference to the activity which uses the fragment for further initialization work.

onCreate() The system calls this method when creating the fragment. You should initialize essential components of the fragment that you want to retain when the fragment is paused or stopped, then resumed.

onCreateView() The system calls this callback when it's time for the fragment to draw its user interface for the first time. To draw a UI for your fragment, you must return a View component from this method that is the root of your fragment's layout. You can return null if the fragment does not provide a UI.

onActivityCreated() The onActivityCreated() is called after the onCreateView() method when the host activity is created. Activity and fragment instance have been created as well as the view hierarchy of the activity. At this point, view can be accessed with the findViewById() method. example. In this method you can instantiate objects which require a Context object

onStart() The onStart() method is called once the fragment gets visible.

onResume() Fragment becomes active.

onPause() The system calls this method as the first indication that the user is leaving the fragment. This is usually where you should commit any changes that should be persisted beyond the current user session.

onStop() Fragment going to be stopped by calling onStop()

onDestroyView() Fragment view will destroy after call this method

onDestroy() onDestroy() called to do final clean up of the fragment's state but Not guaranteed to be called by the Android platform.

This involves number of simple steps to create Fragments.

- First of all decide how many fragments you want to use in an activity. For example let's we want to use two fragments to handle landscape and portrait modes of the device.
- Next based on number of fragments, create classes which will extend the *Fragment* class. The Fragment class has above mentioned callback functions. You can override any of the functions based on your requirements.
- Corresponding to each fragment, you will need to create layout files in XML file. These files will have layout for the defined fragments.
- Finally modify activity file to define the actual logic of replacing fragments based on your requirement.

Types of Fragments

Basically fragments are divided as three stages as shown below.

- **Single frame fragments** – Single frame fragments are using for hand hold devices like mobiles, here we can show only one fragment as a view.
- **List fragments** – fragments having special list view is called as list fragment
- **Fragments transaction** – Using with fragment transaction. we can move one fragment to another fragment.

```
Fragment fr;
if(view == findViewById(R.id.button1)) {
    fr = new FragmentOne();
}
// displaying second fragment
// if button2 is clicked
else {
    fr = new FragmentTwo();
}

FragmentManager fm = getFragmentManager();
// fragment transaction to add or replace
// fragments while activity is running
FragmentTransaction fragmentTransaction = fm.beginTransaction();
fragmentTransaction.replace(R.id.fragment_section, fr);

// making a commit after the transaction
// to assure that the change is effective
fragmentTransaction.commit();
```

How to send data from Activity to Fragment

Sending data from MainActivity to fragment

```
Bundle mBundle = new Bundle();  
mBundle.putString("mText",mEditText.getText().toString());  
mFragment.setArguments(mBundle);  
mFragmentManager.add(R.id.frameLayout, mFragment).commit();
```

Receiving data from MainActivity to fragment

```
// Gets the data from the passed bundle  
Bundle bundle = getArguments();  
String message = bundle.getString("mText");  
  
// Sets the derived data (type String) in the  
// TextView  
myTextView.setText(message);
```

Activity	Fragment
Activity is an application component that gives a user interface where the user can interact.	The fragment is only part of an activity, it basically contributes its UI to that activity.
Activity is not dependent on fragment	Fragment is dependent on activity. It can't exist independently.
we need to mention all activity it in the manifest.xml file	Fragment is not required to mention in the manifest file
We can't create multi-screen UI without using fragment in an activity,	After using multiple fragments in a single activity, we can create a multi-screen UI.
Activity can exist without a Fragment	Fragment cannot be used without an Activity.
Creating a project using only Activity then it's difficult to manage	While Using fragments in the project, the project structure will be good and we can handle it easily.
Lifecycle methods are hosted by OS. The activity has its own life cycle.	Lifecycle methods in fragments are hosted by hosting the activity.
Activity is not lite weight.	The fragment is the lite weight.

ArrayAdapter

The Adapter acts as a bridge between the UI Component and the Data Source. It converts data from the data sources into view items that can be displayed into the UI Component. Data Source can be Arrays, HashMap, Database, etc. and UI Components can be ListView, GridView, Spinner, etc.

ArrayAdapter is the most commonly used adapter in android. When you have a list of single type items which are stored in an array you can use ArrayAdapter. Likewise, if you have a list of phone numbers, names, or cities.

ArrayAdapter has a layout with a single TextView.

The basic syntax for ArrayAdapter is given as:

```
public ArrayAdapter(Context context, int resource, int textViewResourceId, T[] objects)
```

context

The current context. This value can not be null.

resource

The resource ID for the layout file containing a layout to use when instantiating views.

textViewResourceId

The id of the TextView within the layout resource to be populated.

objects

The objects to represent in the ListView. This value cannot be null.

```
String courseList[] = {"C-Programming", "Data Structure", "Database", "Python",  
                        "Java", "Operating System", "Compiler Design", "Android Development"};
```

```
ArrayAdapter arrayAdapter = new ArrayAdapter(this, R.layout.itemListView, R.id.itemTextView,  
courseList[]);
```

SimpleAdapter

In Android, whenever we want to bind some data which we get from any data source (e.g. ArrayList, HashMap, SQLite, etc.) with a UI component(e.g. ListView, GridView, etc.) then **Adapter** comes into the picture.

Basically **Adapter** acts as a bridge between the UI component and data sources.

Here **Simple Adapter** is one type of **Adapter**. It is basically an easy adapter to map static data to views defined in our XML file(UI component) and is used for customization of List or Grid items.

Here we use an ArrayList of Map (e.g. hashmap, mutable map, etc.) for data-backing. Each entry in an ArrayList is corresponding to one row of a list. The Map contains the data for each row. Now to display the row we need a view for which we used to specify a custom list item file (an XML file).

Basic syntax of simpleAdapter:

SimpleAdapter(Context context, List<? extends Map<String, ?>> data, int resource, String[] from, int[] to)

Example:

```
ListView listView;
```

```
// creating a String type array (fruitNames)
// which contains names of different fruits' images
String fruitNames[] = { "Banana", "Grape", "Guava", "Mango", "Orange", "Watermelon" };

// creating an Integer type array (fruitImageIds) which
// contains IDs of different fruits' images
int fruitImageIds[] = {R.drawable.banana,
    R.drawable.grape,
    R.drawable.guava,
    R.drawable.mango,
    R.drawable.orange,
    R.drawable.watermelon};
```

@Override

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
  
    // Binding the ListView of activity_main.xml file  
    // with this java code in MainActivity.java  
    listView = findViewById(R.id.listView);  
  
    // creating an ArrayList of HashMap. The KEY of the HashMap  
    // is a String and VALUE is of any datatype(Object)  
    ArrayList<HashMap<String, Object>> list = new ArrayList<>();  
  
    // By a for loop, entering different types of data in HashMap,  
    // and adding this map including it's datas into the ArrayList  
    // as list item and this list is the second parameter of the SimpleAdapter  
    for (int i = 0; i < fruitNames.length; i++) {  
  
        // creating an Object of HashMap class  
        HashMap<String, Object> map = new HashMap<>();
```

```
// Data entry in HashMap
    map.put("fruitName", fruitNames[i]);
    map.put("fruitImage", fruitImageIds[i]);

    // adding the HashMap to the ArrayList
    list.add(map);
}

// creating A string type array(from) which contains
// column names for each View in each row of the list
// and this array(form) is the fourth parameter of the SimpleAdapter
String[] from = {"fruitName", "fruitImage"};

// creating an integer type array(to) which contains
// id of each View in each row of the list
// and this array(form) is the fifth parameter of the SimpleAdapter
int to[] = {R.id.textView, R.id.imageView};

// creating an Object of SimpleAdapter class and
// passing all the required parameters
SimpleAdapter simpleAdapter = new SimpleAdapter(getApplicationContext(), list,
R.layout.list_row_items, from, to);

// now setting the simpleAdapter to the ListView
listView.setAdapter(simpleAdapter);
}
```